

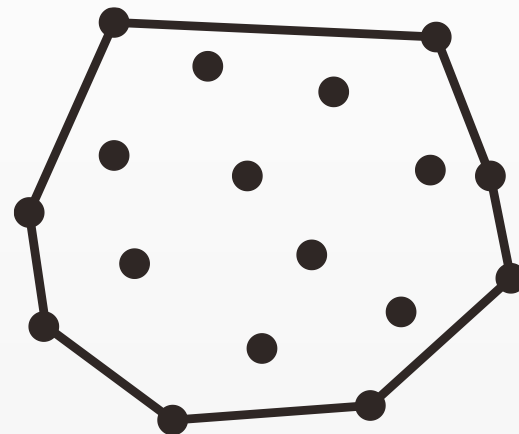
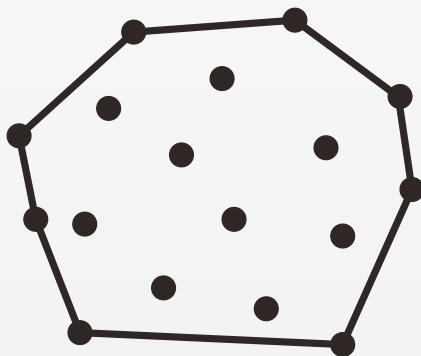


進階程式設計 - 學習歷程

復旦高中 郭昀漢

大綱

- A. 學習目的
- B. 課程簡述
- C. 專題簡介
- D. 報告展示
- E. 心得反思



A. 學習目的

我當初選這個多元選修，是因為我對程式設計非常有興趣，而且我有參加復旦程式設計進階班社團，因此具有一定的程式基礎知識。選擇這堂課，對我來說是加強我演算法的能力，也算是複習之前在社團中不太清楚的知識。老師在課程中帶領學生理解程式設計、資料結構與演算法之間的關聯，除了帶我們認識程式語言架構外，也培養我們探索資訊科技新知的能力；教我們運用程式設計實踐運算思維的解題方法，進而提升學生利用資訊科技知能、創新思考與解決問題的能力。

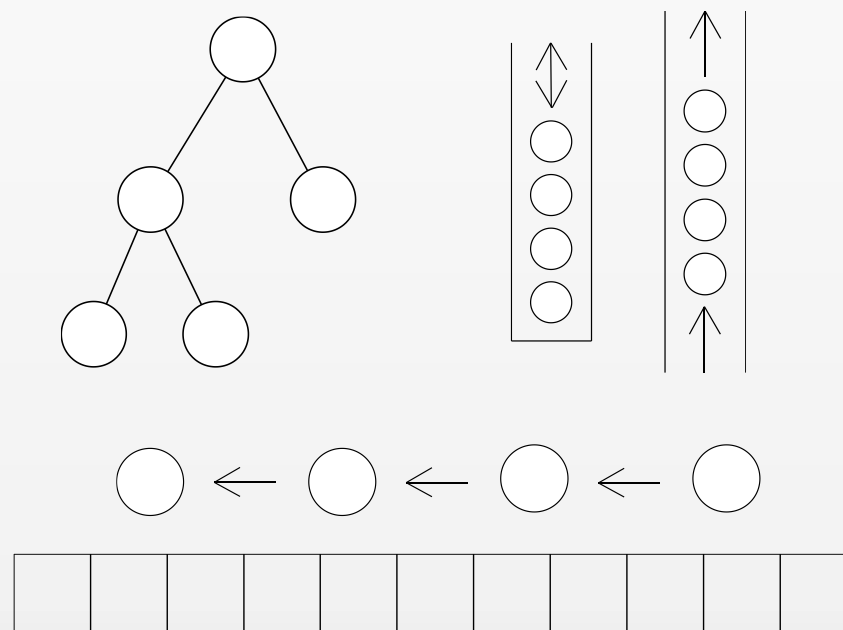


B. 課程簡述

老師會用PPT介紹資料結構與演算法的相關知識，學生了解後便要試著實做出來。而在期末要自己想題目，做出與程式設計相關的專題報告。

課程學習內容：

1. 資料結構導論
2. 陣列結構、鏈結串列
3. 堆疊、佇列
4. 遞迴、樹狀結構
5. 排序、搜尋、圖形架構

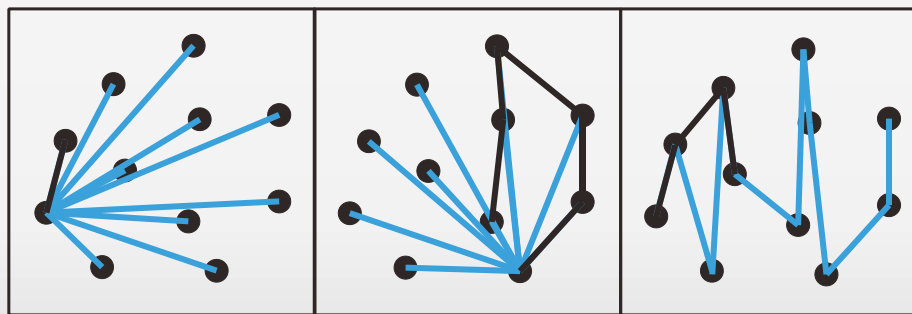


C. 專題簡介

期末報告我選擇研究「凸包」，因為我曾在演算法的書上看過，但未曾實作出來，加上我覺得這個題目不會太難，又具有挑戰性，因此我想嘗試做做看。

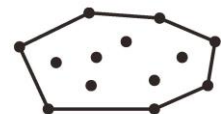
我試了三種凸包演算法，想把這些理論中的策略都實做出來。三種演算法分別是「包裹法 (Gift Wrapping)」、「葛立恆掃描法 (Graham's Scan)」、「安德魯演算法 (Andrew's Algorithm)」，這三個是我查的資料中最常見且我能理解的凸包演算法。

▼三種凸包演算法示意圖



D. 報告展示

(5/1)

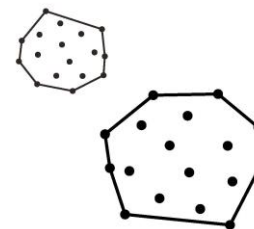


凸包 - Convex Hull

349班 14號 郭昀漢

大綱

- ▶ 凸包定義
- ▶ 二維凸包
- ▶ 應用範圍
- ▶ 演算法介紹
- ▶ 實作程式碼
- ▶ 執行結果
- ▶ 心得感想
- ▶ 資料來源



凸包定義

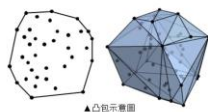
▶ 維基百科：

「在一個實數向量空間 V 中，對於給定集合 X ，所有包含 X 的凸集之交集 S 被稱為 X 的凸包。」

▶ 演算法筆記：

「在高維空間中有一群散佈各處的點，「凸包」是包圍這群點的所有外殼當中，表面積與容積最小的一個外殼，而最小的外殼一定是凸的。」

「凸包是能包圍全部點的
最小凸多邊形或凸殼。」



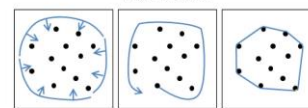
▲凸包示意圖

二維凸包

- ▶ 二維平面上的凸包是凸多邊形，
最外圍的點連線組成的多邊形即是凸包。

- ▶ 可將二維平面凸包視為一條剛好包著所有點的橡皮圈，
也可想像一條線繞最外圍的點。

▼二維平面凸包示意圖



D. 報告展示

(5/2)

應用範圍

- ▶ 圖形處理
- ▶ 電腦視覺
- ▶ 地理資訊系統
- ▶ 無人地面車輛路徑規劃
- ▶ 二次表面巡航導彈任務規劃

▼ 凸包可用於識別物體



演算法介紹

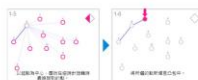
◆ 包裹法 (Gift Wrapping)

▶ 演算法說明：

選定外圍一點作為起點，以起點為中心尋找順或逆時針旋轉時最晚碰到的點，加入此點進入凸包並作為下一次旋轉的起點，重複數次後，回到原點即可得到凸包。

▶ 演算法動畫：

https://www.fq.com.tw/activity/72708/answer/1/algorithm/gift_wrapping/anim.html

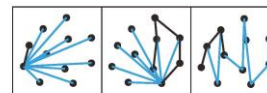


▶ 時間複雜度 $O(\Delta \log N)$

演算法介紹

以下是三種凸包演算法介紹：

- ◆ 包裹法 (Gift Wrapping)
- ◆ 葛立恆掃描法 (Graham's Scan)
- ◆ 安德魯演算法 (Andrew's Algorithm)



演算法介紹

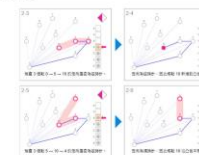
◆ 葛立恆掃描法 (Graham's Scan)

▶ 演算法說明：

選定外圍一點作為起點，以起點為中心按角度排序，依序加入點，並判斷前數三個點的走向是順或逆時針，若反方向就從凸包刪除，當所有點走訪完畢就得到凸包。

▶ 演算法動畫：

https://www.fq.com.tw/activity/72708/answer/1/algorithm/graham_scan/anim.html



▶ 時間複雜度 $O(N \log N)$

(5/3)

```

51         next = i;
52     }
53     if (next == start) break; // 遇到起始點結束
54     v1.push_back(v1[next]); // 記錄此刻的點
55     current = next; // 將下一位設置
56 }
57
58 // 基之選擇方法
59 void gsd (vector<int> &min_element(v1.begin(), v1.end(), compare)) // 尋找最小的點為起點
60 {
61     sort(v1.begin() + 1, v1.end(), compare_angle); // 其餘各點依角度排序
62     for (int i = 0; i < v1.size(); i++) {
63         if (i < 2)
64             v6.push_back(v1[i]);
65         continue;
66     }
67     while (v6.size() >= 2 && cross(v6[v6.size() - 2], v6[v6.size() - 1], v1[i]) <= 0) {
68         v6.pop_back();
69     }
70     v6.push_back(v1[i]);
71 }
72
73 // 安排搜尋算法
74
75

```


D. 報告展示

(5/4)

實作程式碼

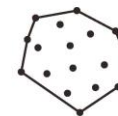
(4/6)

```
76 void as() {
77     sort(v3.begin(), v3.end()); // 點的排序
78     for (int i = 0; i < v3.size(); i++) { // 點上平面
79         point p1 {x = v3[i].first, y = v3[i].second};
80         if (i < 2) {
81             v4.push_back(p1);
82             continue;
83         }
84         while (v4.size() >= 2) {
85             point p2 {x = v4[v4.size() - 1].x, y = v4[v4.size() - 1].y};
86             point p3 {x = v4[v4.size() - 2].x, y = v4[v4.size() - 2].y};
87             if (cross(p2, p3, p1) > 0) break;
88             v4.pop_back();
89         }
90         v4.push_back(p1);
91     }
92     for (int i = v3.size() - 1; i >= 0; i--) { // 點下平面
93         point p1 {x = v3[i].first, y = v3[i].second};
94         if (i > v3.size() - 2) {
95             v5.push_back(p1);
96             continue;
97         }
98         while (v5.size() >= 2) {
99             point p2 {x = v5[v5.size() - 1].x, y = v5[v5.size() - 1].y};
100             point p3 {x = v5[v5.size() - 2].x, y = v5[v5.size() - 2].y};
```

實作程式碼

(5/6)

```
101     if (cross(p2, p3, p1) > 0) break;
102     v5.pop_back();
103     v5.push_back(p1);
104 }
105 }
106 }
107
108 int main() {
109     int n, x, y; // 輸入點座標
110     cout << "輸入點座標的個數: ";
111     cin >> n;
112     if (n == 0) cout << "無點可包//";
113     cout << '\n';
114     for (int i = 0; i < n; i++) {
115         cout << "點 " << i + 1 << " 座標: \n";
116         cout << "x = ";
117         cin >> x;
118         cout << "y = ";
119         cin >> y;
120         point p {x = x, y = y};
121         v1.push_back(p);
122         v3.push_back(make_pair(x, y));
123     }
124     g(); // 包圍法
125 }
```



實作程式碼

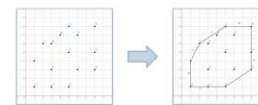
(6/6)

```
126 g() { // 建立包圍法
127     as(); // 點上平面
128     cout << "執行結果: \n"; // 點上平面
129     cout << "使用「包圍法」得到的凸包: \n";
130     for (int i = 0; i < v2.size(); i++) {
131         cout << "(" << v2[i].x << ", " << v2[i].y << ") ";
132         if (i < v2.size() - 1) cout << ", ";
133     }
134     cout << "\n使用「點上平面」得到的凸包: \n";
135     for (int i = 0; i < v6.size(); i++) {
136         cout << "(" << v6[i].x << ", " << v6[i].y << ") ";
137         if (i < v6.size() - 1) cout << ", ";
138     }
139     cout << "\n使用「安德魯演算法」得到的凸包: \n";
140     for (int i = 0; i < v4.size(); i++) {
141         cout << "(" << v4[i].x << ", " << v4[i].y << ") ";
142         if (i < v4.size() - 1) cout << ", ";
143     }
144     if (v5.size() > 2) cout << ", ";
145     for (int i = 1; i < v5.size() - 1; i++) {
146         cout << "(" << v5[i].x << ", " << v5[i].y << ") ";
147         if (i < v5.size() - 2) cout << ", ";
148     }
149     cout << "\n";
150 }
```



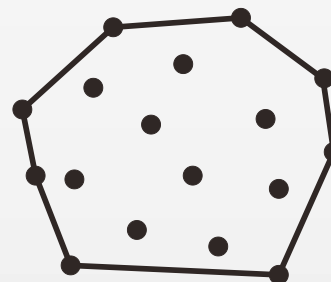
執行結果

- ▶ 輸入:
- 一 點的數量 (例: 15)
 - 二 所有點座標 (例: 111426313336475155586367838588)
- ▶ 輸出:
- 一 使用「包圍法」得到的凸包
 - 二 使用「點上平面」得到的凸包
 - 三 使用「安德魯演算法」得到的凸包



(5/5)

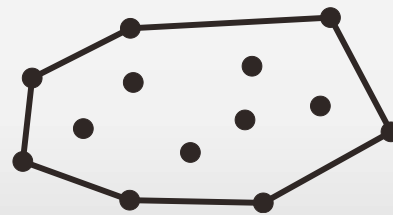
謝謝大家



E. 心得反思

這門課讓我學到許多程式設計的進階知識外，也讓我了解演算法與資料結構的相關運用等不同層面。除此之外，製作專題時所讀的資料；試著把凸包演算法的程式碼寫出來，也都讓我對凸包進一步認識，並且在編寫程式遇到問題時，讓我學習檢討自己的失誤，發覺哪裡寫錯需要改善，不知不覺中，使我進一步的熟悉程式與演算法，並培養自我糾正的能力。

這次的課程教會我許多實用的知識，也讓我在編寫程式無誤時獲得快樂與成就感，期望在未來我可以探索更多有趣的演算法，學習其他程式相關的領域。





THANK YOU